Avance 2 El servicio central y la seguridad

Proyecto 1 Renta de Mobiliario y Equipo para Eventos

En este avance convertí el dominio de renta de mobiliario en un servicio HTTP con persistencia real en MySQL y control de acceso. Conservé las reglas del avance anterior y agregué validación de entradas, migraciones, autenticación, permisos y documentación interactiva para que otras aplicaciones puedan consumirlo.

Desarrollo del servicio

El dominio continúa separado de HTTP y de la base de datos. Sus cinco archivos se mantienen idénticos al avance 1. El nuevo adaptador implementa RepositorioRenta y las operaciones findById, findAll y save del contrato Repository, por lo que las reglas siguen utilizando las mismas interfaces.

Parte	Responsabilidad
Dominio	Entidades, disponibilidad, cotización, reservas e inventario. Sin consultas SQL ni dependencias del servidor.
Servicio HTTP	27 puntos de entrada agrupados por recurso, validación de datos, respuestas y autorización.
Adaptador MySQL	Traducción a tablas relacionales, transacciones, consultas parametrizadas y resultados independientes.
Composición	Único lugar donde se elige memoria o MySQL. HTTP entrega un pool; la consola anterior sigue usando memoria.
Documentación	Swagger generado de los esquemas de cada ruta y de los contratos del modelo.

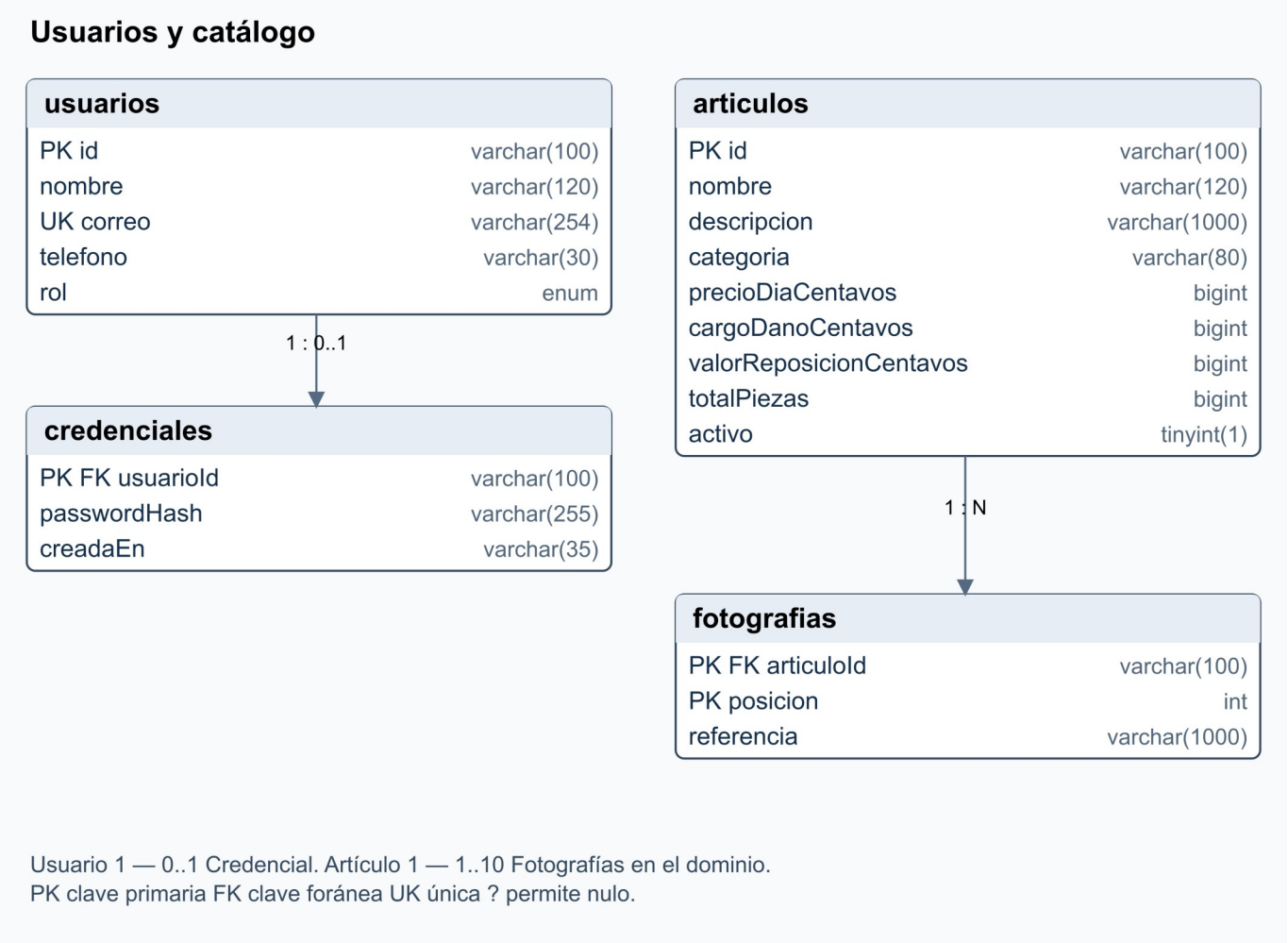
Resultado comprobado

Las 33 pruebas originales del dominio y 73 escenarios de integración con MySQL pasaron. También ejecuté peticiones HTTP reales y comprobé que los datos se conservan después de reiniciar el proceso. La misma operación administrativa responde 401 sin credencial, 403 con un cliente y 201 con un administrador.

El trabajo del servicio está organizado en la rama local feat-servicio. Los datos utilizados en las demostraciones son ficticios.

Modelo de datos de usuarios y catálogo

El esquema completo contiene 11 tablas, 79 columnas y 10 relaciones por clave foránea. Las siguientes vistas se generaron con metadatos consultados en MySQL 8.0.42. PK identifica la clave primaria, FK una referencia, UK una restricción única y el signo de interrogación indica que se admite nulo.



Usuario conserva nombre, correo, teléfono y rol. El correo es único y se normaliza al registrarse. Los roles posibles son administrador y cliente. Una cuenta puede tener una credencial asociada; su clave primaria también es una referencia al usuario.

Artículo almacena los precios en centavos y la existencia total de piezas intercambiables. Las referencias de fotografías se guardan por separado, con orden definido por posición. El dominio exige entre una y diez fotografías y valida que el cargo por daño no supere el valor de reposición.

El esquema relacional completo y los tipos precisos están en docs/modelo-datos.json y docs/modelo-datos.mmd. El archivo de metadatos no contiene usuarios, hashes ni contraseñas.

Modelo de datos de reservas y recolección

Cada reserva pertenece a un usuario cliente. El periodo se almacena como días civiles inclusivos; los instantes reales se conservan en texto ISO. Los detalles guardan una copia de los precios, para que editar el catálogo no cambie una reserva confirmada.



Los estados posibles son apartada, entregada, devuelta y cancelada. La solicitud de cancelación se representa por sus tres campos, que permanecen nulos mientras no exista. Detalles y recolecciones referencian tanto la reserva como el artículo; cada par reserva y artículo es único en su tabla.

La recolección clasifica piezas buenas, dañadas y faltantes. Los importes de renta, garantía, cargos y devoluciones permiten conservar el historial económico sin sobrescribir los precios del detalle.

Modelo de inventario e historial de cambios



Los bloqueos pertenecen a un artículo y pueden vincularse con una reserva cuando proceden de una incidencia. Sus tipos son mantenimiento, dano y faltante. Un ajuste registra el valor anterior, la nueva existencia, el motivo y el instante de la operación. Las reservas y bloqueos comprometidos limitan la reducción de existencias.

Migraciones entregadas

Cambio	Contenido
001_dominio.sql	Crea las entidades del negocio, las tablas hijas y el cerrojo de coordinación.
002_credenciales.sql	Agrega la tabla de credenciales y los índices de consulta por cliente, estado, artículo y periodo.

schema_migrations conserva el nombre, la fecha y la huella SHA256 de cada migración. Repetir la ejecución comprueba el historial y omite lo ya aplicado. El DDL de MySQL realiza confirmaciones implícitas: una migración interrumpida requiere revisar el esquema antes de reintentar; no se afirma que su archivo completo tenga rollback.

Puntos de entrada del servicio

Las rutas están agrupadas y nombradas por recurso. Swagger muestra el cuerpo, los parámetros y el esquema de cada respuesta. La siguiente tabla incluye las 27 entradas de negocio y su resultado de éxito.

Método	Recurso	Permiso	Éxito
GET	/salud	Público	200
POST	/usuarios	Público	201
GET	/usuarios	Administrador	200
POST	/sesiones	Público	200
GET	/usuarios/me	Autenticado	200
GET	/articulos	Público	200
POST	/articulos	Administrador	201
GET	/inventario	Administrador	200
GET	/articulos/{id}	Público	200
PUT	/articulos/{id}	Administrador	200
GET	/articulos/{id}/disponibilidad	Público	200
POST	/cotizaciones	Público	200
POST	/reservas	Cliente propio o administrador	201
GET	/reservas	Cliente propio o administrador	200

Identidad y propiedad de las reservas

El registro público crea exclusivamente clientes y rechaza un campo rol externo. Para un cliente autenticado, la identidad efectiva de la reserva se obtiene de la credencial verificada. Administración debe indicar el cliente al crear una reserva en su nombre. Un cliente no puede abrir ni consultar el historial de otra cuenta.

Los errores posibles incluyen 400, 401, 403, 404, 409, 413, 415 y 500, según corresponda. Los recursos de Swagger y el preflight CORS conservan el formato propio de sus protocolos.

Puntos de entrada de operación e inventario

Las rutas están agrupadas y nombradas por recurso. Swagger muestra el cuerpo, los parámetros y el esquema de cada respuesta. La siguiente tabla incluye las 27 entradas de negocio y su resultado de éxito.

Método	Recurso	Permiso	Éxito
GET	/reservas/{id}	Cliente propio o administrador	200
POST	/reservas/{id}/solicitudes-cancelacion	Cliente propietario	201
POST	/reservas/{id}/cancelacion	Administrador	201
POST	/reservas/{id}/entrega	Administrador	201
POST	/reservas/{id}/recoleccion	Administrador	201
POST	/reservas/{id}/devoluciones-deposito	Administrador	201
POST	/articulos/{id}/ajustes	Administrador	201
GET	/ajustes	Administrador	200
POST	/bloqueos	Administrador	201
GET	/bloqueos	Administrador	200
PATCH	/bloqueos/{id}	Administrador	200
GET	/calendario	Administrador	200
GET	/resumen	Administrador	200

Alcance de las operaciones

La solicitud de cancelación no libera piezas por sí sola. Administración aprueba la cancelación, registra entregas y cobros, clasifica la recolección y registra la devolución de garantía. También administra bloqueos, ajustes, calendario y resumen. No se borran reservas históricas.

Los errores posibles incluyen 400, 401, 403, 404, 409, 413, 415 y 500, según corresponda. Los recursos de Swagger y el preflight CORS conservan el formato propio de sus protocolos.

Validación y formato de las respuestas

Las entradas JSON se comprueban con esquemas antes de llamar al dominio. Se validan campos requeridos, tipos, longitudes, cantidades enteras, fechas civiles reales, límites y propiedades desconocidas. Las cantidades de un cuerpo JSON no se aceptan como texto. Los precios de query se reciben como cadenas numéricas validadas y se convierten explícitamente.

Sobre uniforme

```
{"ok":true,"datos":{"resultado":"ejemplo"},
 "error":null,"solicitudId":"req-1"}
```

```
{"ok":false,"datos":null,
 "error":{"codigo":"SIN_PERMISO",
 "mensaje":"Tu rol no tiene permiso para esta operación."},
 "solicitudId":"req-2"}
```

Estado	Significado
200	Consulta, sesión o actualización correcta.
201	Recurso o evento registrado correctamente.
400	Petición o JSON mal formado; datos que no cumplen el esquema.
401	Credencial ausente, inválida, alterada o vencida.
403	Identidad válida con rol incorrecto o acceso a una reserva ajena.
404	Entidad o ruta inexistente.
409	Regla del dominio que impide la operación, con su código específico.
413 y 415	Cuerpo superior a 64 KiB o tipo de contenido no admitido.
500	Fallo interno; mensaje público genérico sin detalles sensibles.

Todos los rechazos del dominio, salvo las entidades inexistentes, se traducen a conflicto. Por ejemplo, DISPONIBILIDAD_INSUFICIENTE y EXISTENCIAS_COMPROMETIDAS mantienen su significado y responden 409. La bitácora registra método, patrón de ruta, estado y duración en milisegundos de cada petición, sin guardar cuerpos ni credenciales de sesión.

Autenticación y permisos de los roles

Acción	Visitante	Cliente	Administrador
Explorar catálogo y cotizar	Sí	Sí	Sí
Registrar cuenta cliente e iniciar sesión	Sí	Sí	Sí
Consultar perfil propio	No	Sí	Sí
Crear y consultar reservas	No	Solo propias	Todos los clientes
Solicitar cancelación	No	Solo propia	Aprueba cancelación
Catálogo e inventario administrativo	No	No	Sí
Entregas recolecciones y garantías	No	No	Sí
Usuarios calendario y resumen	No	No	Sí

Protección de las credenciales

La contraseña se transforma con scrypt asíncrono antes de guardarse. Cada derivación usa una sal aleatoria de 16 bytes, N igual a 32768, r igual a 8, p igual a 1 y una clave derivada de 64 bytes. La tabla de credenciales conserva parámetros, sal y hash. La comparación utiliza timingSafeEqual; no se guarda la contraseña legible ni se devuelve el hash al cliente.

El inicio de sesión entrega un JWT firmado con HS256 que vence en una hora. Se verifican la firma, el emisor, la audiencia y las declaraciones exp y sub. Después se consulta el rol actual en MySQL. No se confía en un rol enviado por el navegador ni en un identificador de cliente sin autenticación.

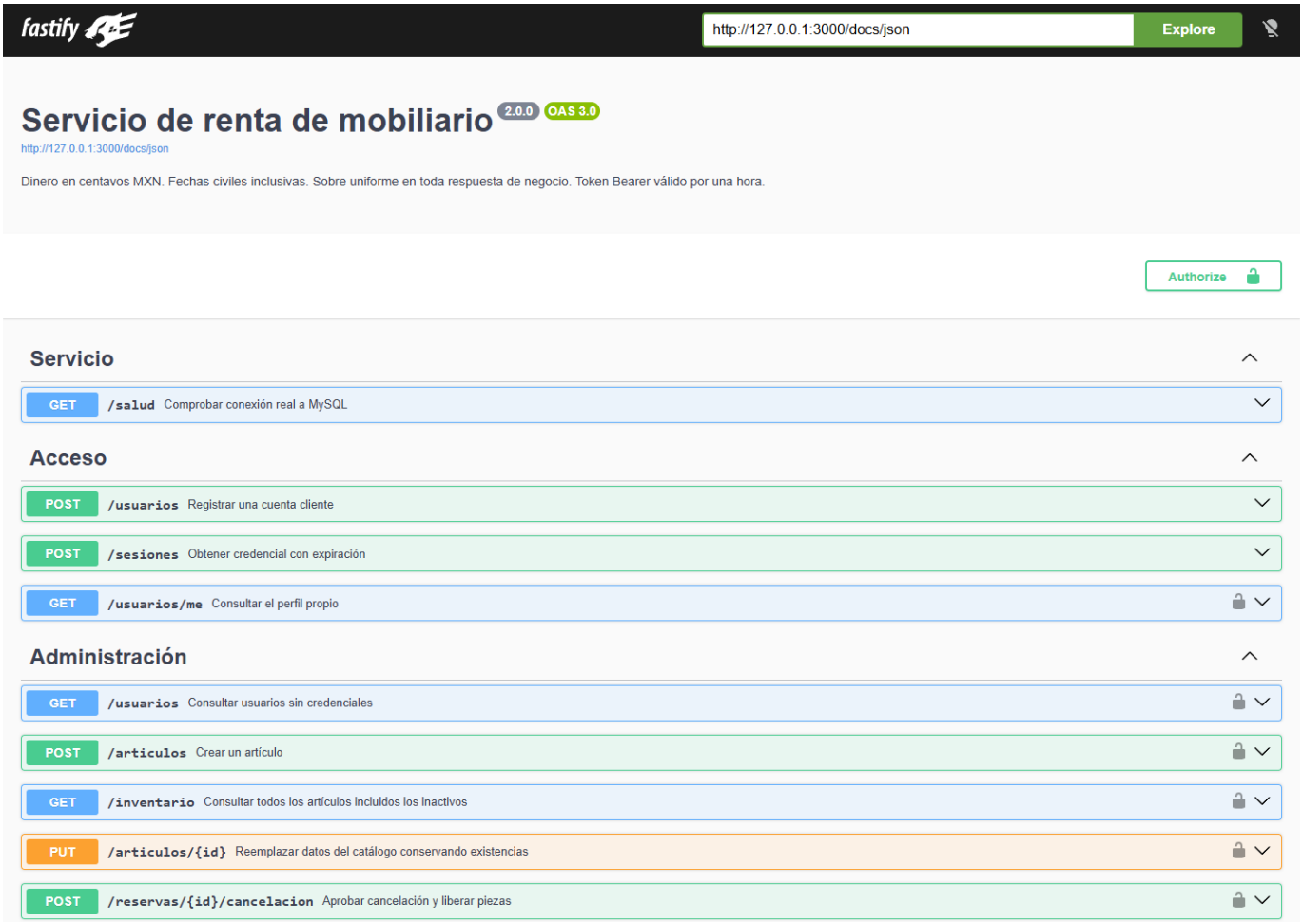
Configuración y acceso desde el navegador

El archivo real .env queda excluido de Git. El ejemplo .env.example contiene los nombres de las variables y marcadores de reemplazo. Las contraseñas de los usuarios de arranque se toman de la configuración local. Los registros ocultan Authorization y cookies.

CORS permite exclusivamente los orígenes configurados para los futuros clientes: http://localhost:5173 y http://localhost:5174. Si cambian sus direcciones, se actualiza la configuración. CORS controla el permiso del navegador; las operaciones protegidas siempre requieren una credencial válida.

Documentación interactiva del servicio

La documentación se genera desde los esquemas declarados en las rutas y se consulta en http://127.0.0.1:3000/docs/. Incluye modelos, parámetros, cuerpos JSON, respuestas y la indicación de seguridad Bearer. Los 14 esquemas del modelo se generan desde el árbol de sintaxis de TypeScript, conservando intacto el dominio.



Probé GET /articulos mediante Try it out en Swagger y obtuve HTTP 200. La comprobación quedó registrada en swagger-verificado.json y en una captura adicional. El contrato exportado está en docs/openapi.json; el contrato vivo se genera en /docs/json a partir del código que está ejecutándose.

Transacciones y almacenamiento

El adaptador usa una conexión por unidad de trabajo, aislamiento SERIALIZABLE y un cerrojo de fila compartido. Confirma con COMMIT o revierte con ROLLBACK. El cerrojo coordina conexiones y procesos distintos. Las tablas hijas se actualizan dentro de la misma transacción que su reserva o artículo.

Evidencia del control de acceso

Comparé POST /articulos con el mismo cuerpo y tres condiciones de acceso. Sin credencial devuelve 401; con un cliente, 403; con un administrador, 201 y el artículo queda guardado. También alteré manualmente el contenido de una credencial y el servicio la rechazó con 401.



Figura elaborada a partir de las respuestas HTTP reales guardadas en evidencias/avance2/demo-http.json. Es un resumen legible de la ejecución, no una captura de terminal. No contiene contraseñas ni tokens.

Evidencia de los conflictos de negocio

Reservé 80 de 100 sillas durante tres días, a $15 por pieza y día. La renta fue de $3,600, la garantía de $720 y el total inicial de $4,320. La segunda solicitud de 30 piezas y la reducción a 70 existencias fueron rechazadas con HTTP 409. La disponibilidad final conservó 20 piezas libres.



Respuestas HTTP del servicio local

Resumen generado a partir de las peticiones reales guardadas en demo-http.json. No contiene contraseñas ni tokens.

4 Reserva aceptada de 80 piezas por tres días

HTTP 201

POST /reservas


```
{
  "ok": true,
  "datos": {
    "periodo": {
      "desde": "2026-10-30",
      "hasta": "2026-11-01"
    },
    "dias": 3,
    "rentaCentavos": 360000,
    "depositoCentavos": 72000,
    "totalInicialCentavos": 432000,
    "estado": "apartada"
  },
  "error": null
}
```


5 Sobreventa rechazada por el dominio

HTTP 409

POST /reservas


```
{
  "ok": false,
  "datos": null,
  "error": {
    "codigo": "DISPONIBILIDAD_INSUFICIENTE",
    "mensaje": "El artículo A-e76af252-5785-4cee-bac2-94dff97ad5fd
tiene 20 piezas libres; se solicitaron 30."
  },
  "solicitudId": "req-8"
}
```


6 Reducción que afecta compromisos

HTTP 409

POST /articulos/A-e76af252-5785-4cee-bac2-94dff97ad5fd/ajustes


```
{
  "ok": false,
  "datos": null,
  "error": {
    "codigo": "EXISTENCIAS_COMPROMETIDAS",
    "mensaje": "No se pueden dejar menos de 80 piezas: hay reservas
o bloqueos comprometidos."
  },
  "solicitudId": "req-9"
}
```


8 Disponibilidad persistida

HTTP 200

GET /articulos/A-e76af252-5785-4cee-bac2-94dff97ad5fd/disponibilidad?
desde=2026-10-30&hasta=2026-11-01


```
{
  "ok": true,
  "datos": {
    "articuloId": "A-e76af252-5785-4cee-bac2-94dff97ad5fd",
    "totalPiezas": 100,
    "maximoOcupadas": 80,
    "disponibles": 20,
    "deficit": 0
  },
  "error": null,
  "solicitudId": "req-b"
}
```


Los importes de las respuestas se expresan en centavos. La figura resume peticiones reales y sus códigos de negocio. Una prueba adicional detuvo y reinició el proceso: la misma consulta mantuvo 80 piezas ocupadas y 20 disponibles en MySQL.

Comprobaciones e instalación del servicio

Comprobación	Resultado
Pruebas originales del dominio	33 pruebas aprobadas.
Integración con MySQL real	73 escenarios aprobados, dentro de una prueba integral.
Concurrencia entre dos conexiones	Una reserva se confirma y la que excede disponibilidad responde 409.
Rollback y copias	Una operación fallida no deja cambios; modificar un resultado no altera la base.
Migraciones y datos de arranque	Se ejecutaron dos veces sin duplicar los datos iniciales.
Dominio conservado	Sus cinco archivos coinciden con el avance 1.
Swagger y reinicio	Consulta ejecutada desde la interfaz y persistencia comprobada tras reiniciar.
Dependencias de producción	npm audit no reportó vulnerabilidades en la comprobación realizada.

Cómo levantarlo desde cero

Se requiere Node.js 22.18 o posterior y MySQL 8.0.16 o posterior con InnoDB. Después de instalar dependencias se copia .env.example a .env y se configuran la conexión local, un secreto aleatorio, los orígenes y dos contraseñas de arranque. El README incluye una alternativa con Docker Compose, no utilizada para la verificación de esta entrega.

```
npm ci
npm run db:crear
npm run db:migrar
npm run db:sembrar
npm start
```

La siembra prepara admin@example.test, cliente@example.test, cien sillas y veinte mesas. No reinicia contraseñas existentes. Para repetir las comprobaciones se ejecutan npm test y npm run test:integracion. La integración crea una base temporal con nombre aleatorio y elimina únicamente esa base al terminar.

La implementación prioriza corrección para el volumen escolar: serializa las unidades de trabajo y carga entidades mediante el contrato existente. Un volumen mayor requeriría optimizar consultas y particionar la coordinación. El servicio se inicia en la interfaz local.

Referencias

Proyecto 1 Renta de Mobiliario y Equipo para Eventos y Avance 2 El Servicio Central y la Seguridad, guías del profesor. Documentación oficial: fastify.dev/docs/latest/Reference/Validation-and-Serialization/; github.com/fastify/fastify-jwt; github.com/fastify/fastify-cors; github.com/fastify/fastify-swagger; sidorares.github.io/node-mysql2/docs; nodejs.org/api/crypto.html.